

ALGORITHMS II

END-OF-TERM PROJECT

Comparative Algorithm Design and Complexity Analysis

Efficiently Answering Shortest-Path Queries on Weighted Graphs

Project Group (4 Members)

Full Name	Student ID
Faruk Emre Afşar	241401003
Murat Emre Öztürk	241401700
Berfin Güneri	231401058
Berkay Bıyık	231401010

Experiment profile: full | Results date: 2026-05-14T14:58:27 | Python 3.12.3

Spring Semester 2025–2026

Table of Contents

1. Introduction and Problem Definition
 2. Graph Representations and Memory Costs
 3. The Four Implemented Approaches
 4. Theoretical Complexity Analysis
 5. Experimental Evaluation
 6. Comparative Discussion
 7. Conclusion
- References
- Appendix A: Running the Code

1. Introduction and Problem Definition

1.1. The Problem

The core problem addressed in this project is efficiently answering shortest-path queries on a weighted graph. Given a directed graph $G = (V, E)$ and a weight function $w: E \rightarrow \mathbb{R}$ defined on the edges, the shortest path between two nodes s (source) and t (target) is the path from s to t with the smallest total edge weight. The goal is not merely to find the correct answer; it is to solve the same problem with different algorithmic decisions and to understand the impact of those decisions on performance, scalability, and resource usage.

The problem arises in two natural forms: single-source shortest paths (SSSP) — from one source to all nodes; and all-pairs shortest paths (APSP) — between every pair of nodes. Of the four approaches implemented in this project, three solve SSSP and one solves APSP; this directly exposes one of the trade-offs at the heart of the comparison (preprocessing vs. query).

1.2. The Notion of a "Query"

To make the comparison fair, we define a "query" precisely: a query is the cost of computing shortest-path information from one source given that the graph is already built. For single-source algorithms (the two versions of Dijkstra, Bellman-Ford) this is one SSSP run. For Floyd-Warshall, even a single (s, t) query requires the all-pairs preprocessing; once preprocessing is done, every (s, t) lookup is $O(1)$. This definition makes Floyd-Warshall disadvantaged for a single query and advantaged for many queries — which is exactly the core trade-off we study.

1.3. Assumptions

The project brief permits reasonable assumptions as long as they are stated and justified explicitly. The assumptions used in this work are as follows:

- **Weights:** Edge weights are by default non-negative integers (the 1–1000 range, mimicking real road distances). This is Dijkstra's validity condition.
- **Negative weights:** The negative-weight case is also handled separately. Negative weights are generated on a directed acyclic graph (DAG), so no negative cycle can form and the shortest path is well-defined. Negative-cycle detection has also been implemented and tested.
- **Density:** Graph density is not assumed fixed; it is deliberately made an experimental variable (Experiment B). Sparse graphs contain $m \approx 4 \cdot V$ edges, dense graphs $m \approx p \cdot V^2$.
- **Directionality:** The graph is assumed directed. Undirected graphs are modeled by adding each edge in both directions; the real dataset (DIMACS) already provides both directions as separate arcs.

1.4. Why Four Approaches?

Four approaches were chosen so as to cover all three dimensions required by the brief (graph representation, algorithmic strategy, design philosophy). They also map naturally onto a four-member group structure, with one approach per member:

Table 1: Distribution of the four approaches across representation and philosophy. All three representation forms and four distinct strategies are covered.

#	Approach	Representation	Design philosophy
1	Dijkstra + Binary Heap	Adjacency list	On-demand, time-optimized
2	Dijkstra + Array Scan	Adjacency matrix	On-demand, dense-graph-optimized
3	Bellman-Ford	Edge list	Generality (negative weights), simplicity
4	Floyd-Warshall	Adjacency matrix	Heavy preprocessing, $O(1)$ query (APSP)

This distribution enables meaningful pairwise comparisons: Approach 1 vs. 2 contrasts the behavior of the same algorithm with a different data structure; Approach 1 vs. 3 contrasts different strategies on the same SSSP problem; and Approach 1 vs. 4 contrasts on-demand computation against heavy preprocessing.

2. Graph Representations and Memory Costs

The graph representation — a design decision independent of the algorithmic strategy — determines both memory usage and the cost of the fundamental operations (iterating over neighbors, querying edge existence). Three classic representations are used in this project.

2.1. Edge List

The graph is stored as a list of (u, v, w) triples. The memory cost is $O(E)$. The operation "iterate over all edges in order" is natural and efficient; this makes it the natural input for Bellman-Ford. However, the question "who are the neighbors of node u ?" requires scanning the entire list — which is why it is unsuitable for Dijkstra.

2.2. Adjacency List

For each node u , a list of the outgoing (v, w) edges from u is kept. The memory cost is $O(V + E)$. The question "the neighbors of u " is answered in steps proportional only to u 's degree. On sparse graphs it is the most efficient representation in terms of both memory and access; the heap-based version of Dijkstra uses it.

2.3. Adjacency Matrix

A $V \times V$ matrix is kept; $M[u][v]$ is the weight of the $u \rightarrow v$ edge (∞ if there is no edge). The question "is there a $u \rightarrow v$ edge?" is answered in $O(1)$, and row/column operations on the matrix can be vectorized. However, the memory cost is $O(V^2)$, independent of the number of edges. It is suitable for dense graphs and Floyd-Warshall; but on large sparse graphs it is unacceptably expensive — one of the most striking practical findings of this project comes from here (see Section 5.5).

Table 2: Measured memory footprint of the three representations (sparse graphs, $m \approx 4 \cdot V$). The adjacency matrix grows as $O(V^2)$ while the list-based representations stay $O(V+E)$; the gap widens rapidly as V grows.

V	E	Edge list	Adjacency list	Adjacency matrix	Matrix / List ratio
100	400	35.6 KB	39.9 KB	78.2 KB	2×
200	800	71.6 KB	79.8 KB	312.6 KB	4×
400	1,600	151.9 KB	167.9 KB	1.2 MB	7×
800	3,200	315.3 KB	351.0 KB	4.9 MB	14×
1,600	6,400	644.2 KB	714.0 KB	19.5 MB	28×
3,200	12,800	1.3 MB	1.4 MB	78.1 MB	56×
6,400	25,600	2.6 MB	2.8 MB	312.5 MB	111×
12,800	51,200	5.1 MB	5.7 MB	1250.0 MB	221×

Memory footprint of the graph representations
The adjacency matrix grows fast as $O(V^2)$; on sparse graphs the edge/adjacency list is far more economical

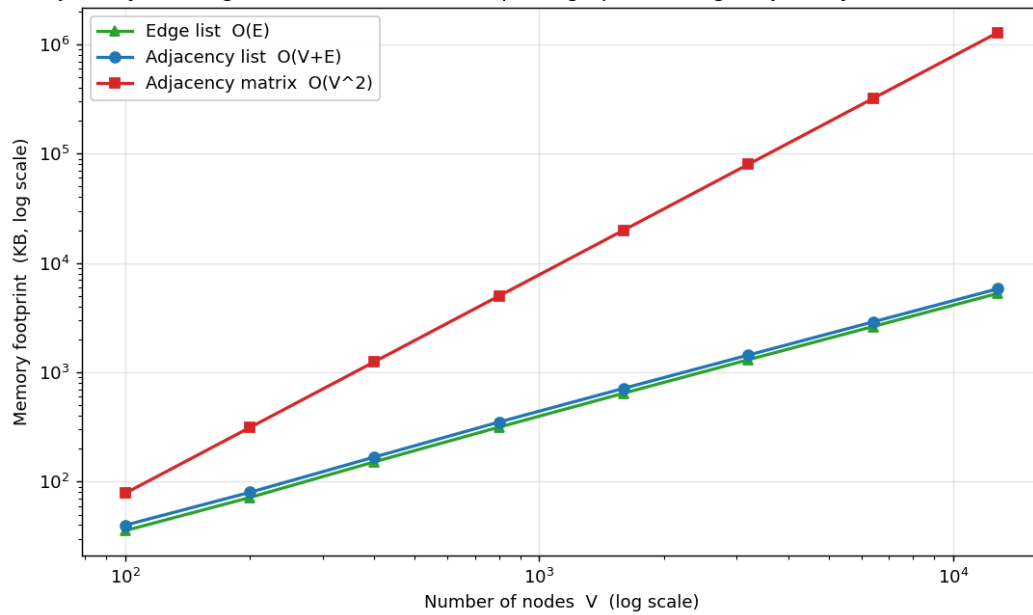


Figure 1: Memory footprint of the graph representations as a function of the number of nodes (log-log). The adjacency matrix has slope ≈ 2 ($O(V^2)$), the list-based representations have slope ≈ 1 ($O(V+E)$).

Conclusion: the choice of representation can by itself determine at which graph sizes an algorithm is feasible. This is seen concretely in Section 5.4 on the 264-thousand-node real road network.

3. The Four Implemented Approaches

This section gives each approach's core idea, the representation it uses, its design philosophy, and its pseudocode. The detailed complexity derivation is in Section 4, and the experimental behavior in Section 5.

3.1. Approach 1 — Dijkstra + Binary Heap + Adjacency List

Dijkstra's algorithm computes single-source shortest paths with a greedy strategy under the assumption of non-negative weights. At each step, the node closest to the source among the not-yet-finalized nodes is selected, "finalized," and its neighbors are relaxed (relaxation). The key observation: because the weights are non-negative, the distance of the node with the smallest tentative distance can no longer be improved.

In this version the query "the node with the smallest tentative distance" is answered in $O(\log V)$ with a binary heap. The "lazy deletion" technique is used: when a node's distance improves, a new entry is pushed onto the heap, and stale entries are skipped when popped. Philosophy: on-demand computation and time optimization. The early-stopping option (stop when the target node is popped from the heap) provides additional speedup for single-pair queries.

```
DIJKSTRA-HEAP(G, source s):
  for each v: dist[v] ← ∞;   dist[s] ← 0
  heap ← { (0, s) }
  while heap not empty:
    (d, u) ← pop minimum from heap
    if u already finalized: continue      # lazy deletion
    finalize u
    for each edge (u → v, w):            # relaxation
      if d + w < dist[v]:
        dist[v] ← d + w;   push (dist[v], v) onto heap
```

3.2. Approach 2 — Dijkstra + Array Scan + Adjacency Matrix

The same greedy Dijkstra strategy, this time implemented without any priority queue. The "not-yet-finalized node with the smallest tentative distance" is found at each step by a linear scan of the distance array ($O(V)$). Relaxation is done over the corresponding row of the adjacency matrix. V steps $\times O(V) = O(V^2)$; this cost is independent of the number of edges.

Philosophy: structural simplicity and optimization for dense graphs. On dense graphs ($E \approx V^2$), $O(V^2)$ is better than the heap version's $O(E \log V)$. There is also no data-structure overhead (heap operations). The cost: $O(V^2)$ memory and waste on sparse graphs. In the implementation the min-finding step is vectorized with NumPy; this only lowers the constant factor, it does not change the $O(V^2)$ complexity.

```

DIJKSTRA-ARRAY(M, source s):
    # M: adjacency matrix
    for each v: dist[v] ← ∞;    dist[s] ← 0
    repeat V times:
        u ← not-yet-finalized node with smallest dist    # O(V) scan
        if dist[u] = ∞: break
        finalize u
        for each v:
            # relax over row M[u]
            if dist[u] + M[u][v] < dist[v]:
                dist[v] ← dist[u] + M[u][v]

```

3.3. Approach 3 — Bellman-Ford + Edge List

Bellman-Ford does not need Dijkstra's greedy assumption. Since a shortest path can contain at most $V-1$ edges, if all edges are relaxed repeatedly over $V-1$ passes, every node's distance is finalized. This "brute-force relaxation" approach also works correctly with negative weights, and if one more pass of relaxation is still possible it detects a negative cycle.

Philosophy: generality and simplicity — correctness before speed. The edge list is the natural representation because it maps exactly onto the "iterate over all edges" operation; no adjacency matrix is needed, so memory is $O(V + E)$. An early-stopping check has been added to the implementation: if no relaxation occurs in a pass the algorithm stops; on sparse/random graphs this brings the number of passes in practice well below $V-1$.

```

BELLMAN-FORD(edges E, source s):
    for each v: dist[v] ← ∞;    dist[s] ← 0
    repeat V-1 times:
        changed ← false
        for each edge (u → v, w):
            if dist[u] + w < dist[v]:
                dist[v] ← dist[u] + w;    changed ← true
        if changed = false: break    # early stopping
    for each (u → v, w):
        # negative-cycle check
        if dist[u] + w < dist[v]: "negative cycle exists"

```

3.4. Approach 4 — Floyd-Warshall + Adjacency Matrix

Floyd-Warshall computes shortest paths between all pairs of nodes rather than from a single source. The dynamic-programming idea is this: for each intermediate node k , does the path from i to j get shorter if it goes through k ? When this check is done for all (i, j) pairs and all k , $D[i][j]$ converges to the shortest distance.

Philosophy: heavy preprocessing, very fast queries. $O(V^3)$ is spent once; afterwards every (s, t) query is just a matrix lookup — $O(1)$. It works with negative weights; it detects a negative cycle as a negative value on the diagonal. The cost: $O(V^2)$ memory and tremendous waste for a single query. In the implementation the innermost two loops are vectorized with NumPy; the $O(V^3)$ from the three-level structure does not change (a pure-Python reference version is also present in the code).


```

FLOYD-WARSHALL(M):                                # M: adjacency matrix → D
D ← M
for k = 0 .. V-1:                                  # intermediate node
  for i = 0 .. V-1:
    for j = 0 .. V-1:
      if D[i][k] + D[k][j] < D[i][j]:
        D[i][j] ← D[i][k] + D[k][j]
# D[i][j] = i→j shortest distance; D[i][i] < 0 ⇒ negative cycle

```

3.5. Visualizing the Algorithms' Behavior

The way the approaches "solve the same problem differently" has been visualized step by step on a grid graph. The frames below are taken from the simulations produced by `src/animate.py`, the full animations are available as GIFs in the `results/animations/` folder and can be played.

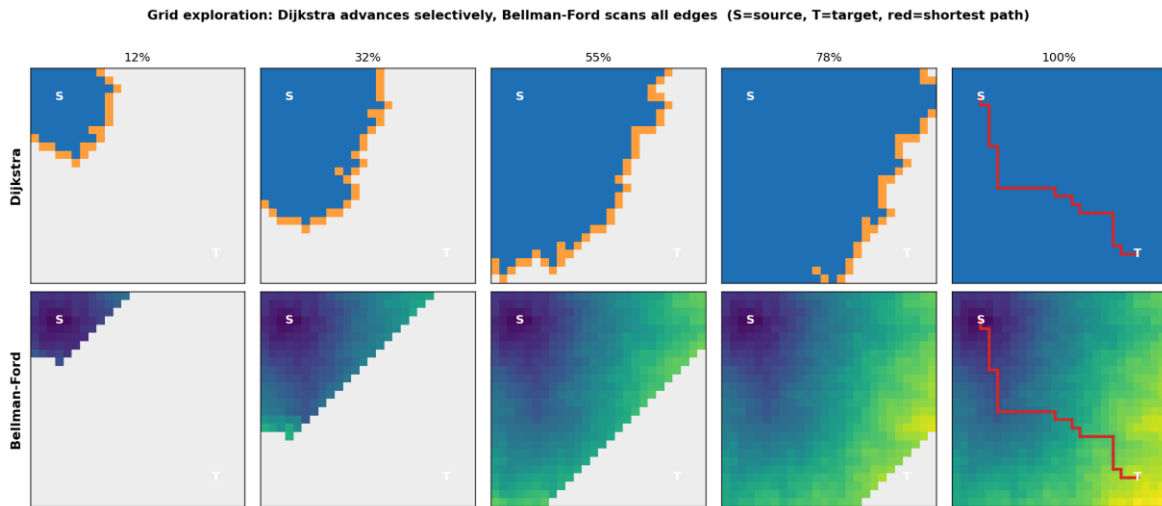


Figure 2: Dijkstra and Bellman-Ford exploring the same grid graph (12% → 100% progress). Dijkstra (top row): the blue region finalizes outward from the source IN ORDER OF DISTANCE; the orange is the thin "frontier" being worked on. Bellman-Ford (bottom row): each pass relaxes all edges INDISCRIMINATELY; the distance field spreads like a wave.

Both algorithms find the same shortest path (red); the difference is IN THE WORK DONE. Dijkstra advances selectively toward the target and works only on a thin frontier; Bellman-Ford rescans all edges in every pass. This is the direct visual counterpart of the operation-count measurements in Section 5.7 (Dijkstra $\approx E$, Bellman-Ford $\approx \text{passes} \times E$).

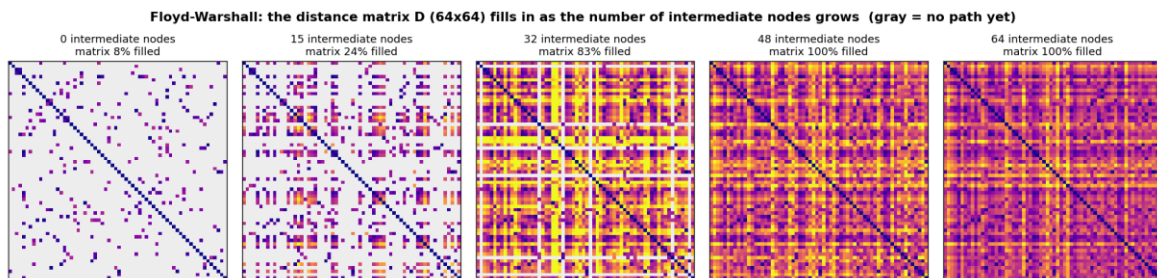


Figure 3: Floyd-Warshall's all-pairs distance matrix D "fills in" as the number of intermediate nodes used increases (gray = no path yet). Instead of a single-source graph exploration, all node pairs are computed at once via dynamic programming — an entirely different computation paradigm.

4. Theoretical Complexity Analysis

This section derives the time and memory complexity of the four approaches and explains how the graph's size (V , E) and structure (density, diameter) affect performance.

Table 3: Theoretical complexity summary of the four approaches. For single-source algorithms a "query" is one SSSP run; for Floyd-Warshall it is an $O(1)$ matrix lookup after $O(V^3)$ preprocessing.

Approach	Preprocessing	Query (time)	Memory	Negative weights
Dijkstra + Binary Heap	—	$O((V + E) \log V)$	$O(V + E)$	No
Dijkstra + Array Scan	—	$O(V^2)$	$O(V^2)$	No
Bellman-Ford	—	$O(V \cdot E)$	$O(V + E)$	Yes (+ cycle detection)
Floyd-Warshall	$O(V^3)$	$O(1)$ lookup	$O(V^2)$	Yes (+ cycle detection)

4.1. Dijkstra + Binary Heap — $O((V + E) \log V)$

Each node is finalized from the heap at most once. Each edge is relaxed at most once, and a relaxation may cause one heap push; hence the total number of pushes is $O(E)$. Since each heap operation (push/pop) is $O(\log V)$, the total is $O((V + E) \log V)$. Memory is $O(V + E)$: the adjacency list is $O(V + E)$, the dist/pred arrays are $O(V)$, and the heap (because of lazy deletion) is $O(E)$. On sparse graphs ($E = O(V)$) this drops to $O(V \log V)$; this is where the algorithm's strength lies.

4.2. Dijkstra + Array Scan — $O(V^2)$

V steps are performed; at each step an $O(V)$ linear scan is done for min-finding and an $O(V)$ relaxation over a matrix row: $V \cdot O(V) = O(V^2)$. This cost is entirely independent of the number of edges. Memory is $O(V^2)$ (the adjacency matrix). Conclusion: on dense graphs ($E = \Theta(V^2)$) this version is faster than the heap version because $O(V^2) \leq O(E \log V)$; on sparse graphs, however, since $O(V^2) \gg O(V \log V)$, it is much slower.

4.3. Bellman-Ford — $O(V \cdot E)$

In the worst case $V-1$ passes are performed and each pass relaxes all E edges: $O(V \cdot E)$. On sparse graphs this is $O(V^2)$, on dense graphs $O(V^3)$. Memory is $O(V + E)$ (only the edge list + arrays; no matrix). Early stopping brings the number of passes in practice down to the order of the number of edges on the shortest paths (the graph's diameter); for this reason the empirical performance is markedly better than the worst case on most random graphs. Even so, the asymptotic upper bound is worse than Dijkstra's; Bellman-Ford's real value is not speed but its ability to solve negative weights correctly.

4.4. Floyd-Warshall — $O(V^3)$ preprocessing, $O(1)$ query

The three nested loops (k, i, j) directly give $O(V^3)$. Once preprocessing is done, any (s, t) query is a $D[s][t]$ matrix lookup: $O(1)$. For Q queries the total cost is $O(V^3 + Q)$; comparing this with Dijkstra's $O(Q \cdot E \log V)$ cost for Q queries, Floyd-Warshall becomes profitable for sufficiently large Q (the break-even point in Experiment C). Memory is $O(V^2)$, which makes the algorithm entirely infeasible on large sparse graphs.

4.5. The Effect of Graph Size and Structure

- **Number of nodes V :** The cost of all approaches increases with V ; but the growth rate differs greatly: Dijkstra-Heap $\approx V \log V$, Dijkstra-Array and sparse Bellman-Ford $\approx V^2$, Floyd-Warshall $\approx V^3$.
- **Number of edges E (density):** The cost of Dijkstra-Heap and Bellman-Ford increases directly with E ; Dijkstra-Array and Floyd-Warshall, however, are entirely independent of E . This forms the basis of the density experiment (Experiment B).
- **Structure (diameter, hubs):** On large-diameter graphs (e.g. road networks, grids) Bellman-Ford's early stopping kicks in later and Dijkstra opens more nodes; on small-diameter (hub-heavy) graphs the opposite holds. Although structure does not change the asymptotic class, it determines the constant factors and the early-stopping behavior.

5. Experimental Evaluation

Correctness alone is not enough; this section measures the empirical behavior of the four approaches and compares it with theoretical expectations. All results are reproducible: fixed seed 20252, experiment profile "full".

5.1. Experimental Setup and Measurement Method

- **Common infrastructure:** All algorithms operate on a single shared Graph object; each requests only the representation it uses. This makes the comparison fair.
- **Time:** Time is measured with `time.perf_counter`. For each (algorithm, graph) an untimed warm-up run is performed; then measurements are taken from multiple source nodes and multiple repetitions, and the median is reported.
- **Memory:** Peak memory is measured with `tracemalloc`. Since the representation is built beforehand, outside the measurement, the reported value is the peak usage of the algorithm's own working structures (heap, arrays, matrices).
- **Graphs:** Synthetic graphs (sparse/dense random) are used to fully control the experiment size; the real DIMACS New York road network is used to see real structure.
- **Environment:** Measurements were done single-threaded, in pure Python + NumPy (Python 3.12.3). Absolute times are hardware-dependent; what matters is the relative behavior and slopes between the approaches.

An important design decision: when an algorithm becomes infeasible on larger inputs because of its theoretical complexity or a memory limit, this is not an error but itself an experimental result, and it is marked as "infeasible" in the plots.

5.2. Experiment A — Scaling with the Number of Nodes

On sparse graphs ($m \approx 4 \cdot V$), the per-query time was measured as the number of nodes V grows. On the log-log axes of the figure, the slope of a power law corresponds to the empirical exponent.

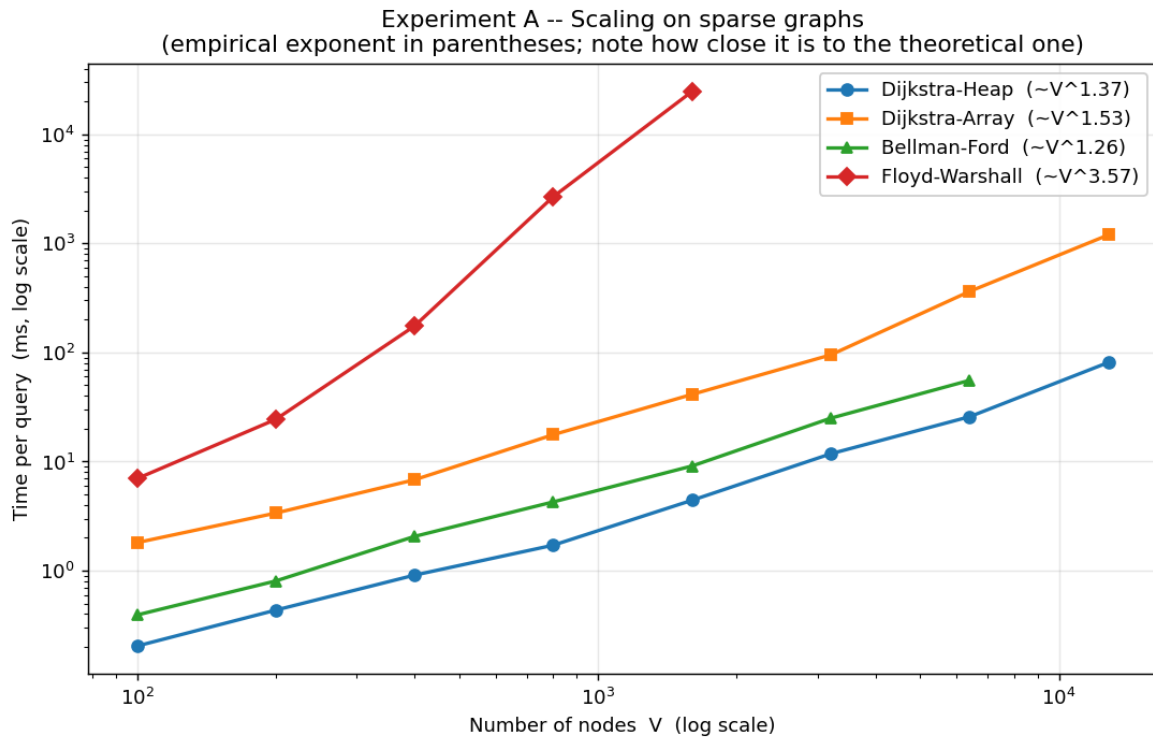


Figure 4: Experiment A: query time vs. V on sparse graphs (log-log). The steeper the curve, the worse the algorithm scales. The empirical exponents in parentheses are fitted to the asymptotic tail (see Section 5.7).

Findings: The ordering of the curves exactly confirms the theoretical complexity ordering. At $V = 1600$ nodes — the largest common size for which all four algorithms produced data — Dijkstra-Heap answered a query in 4.42 ms, Dijkstra-Array in 41.32 ms, Bellman-Ford in 9.11 ms, and Floyd-Warshall in 24.79 s — that is, Floyd-Warshall is roughly 5,610× slower than Dijkstra-Heap. Because of its $O(V^3)$ growth, Floyd-Warshall reaches the "infeasible" boundary earliest ($V > 2,500$); Dijkstra-Array and Bellman-Ford are in between. Dijkstra-Heap, on the other hand, scales up to $V = 12,800$ nodes and completes a query in 81.16 ms.

5.3. Experiment B — The Effect of Edge Density

The number of nodes was kept fixed at $V = 1000$ and the edge density was increased. This experiment directly shows which approaches are sensitive to density.

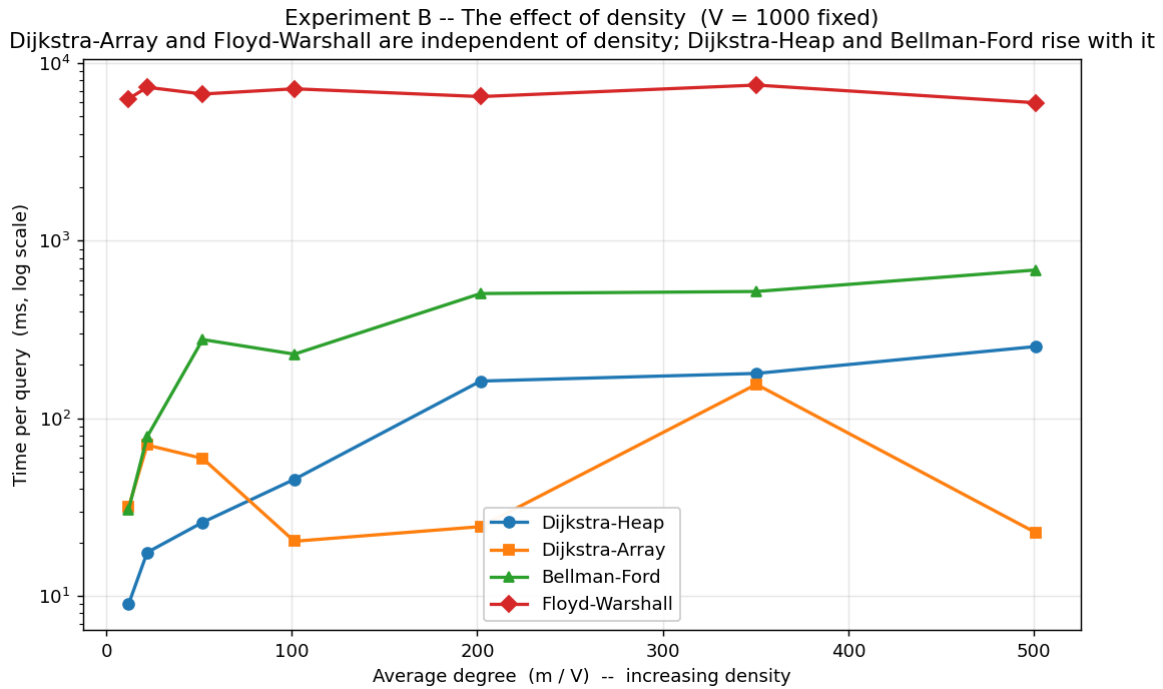


Figure 5: Experiment B: fixed V , increasing density. The time of Dijkstra-Array and Floyd-Warshall is flat (independent of density); Dijkstra-Heap and Bellman-Ford rise with density.

Findings: Since the cost of Dijkstra-Array and Floyd-Warshall depends only on V , their curves are flat. Dijkstra-Heap's $O(E \log V)$ cost and Bellman-Ford's $O(V \cdot E)$ cost, however, rise with density. Conclusion: at the sparse end Dijkstra-Heap is well ahead; as density increases Dijkstra-Array catches up to it and overtakes it — that is, the answer to the question "which is the best algorithm" depends on the graph's density. This is the concrete case that explains why the adjacency-matrix representation exists.

5.4. Experiment C — The Preprocessing / Query-Count Trade-off

On a fixed graph ($V = 1000$), the total running time was studied as a function of the query count Q . Single-source algorithms run from scratch on every query; Floyd-Warshall does $O(V^3)$ preprocessing once, after which each query is $\sim O(1)$.

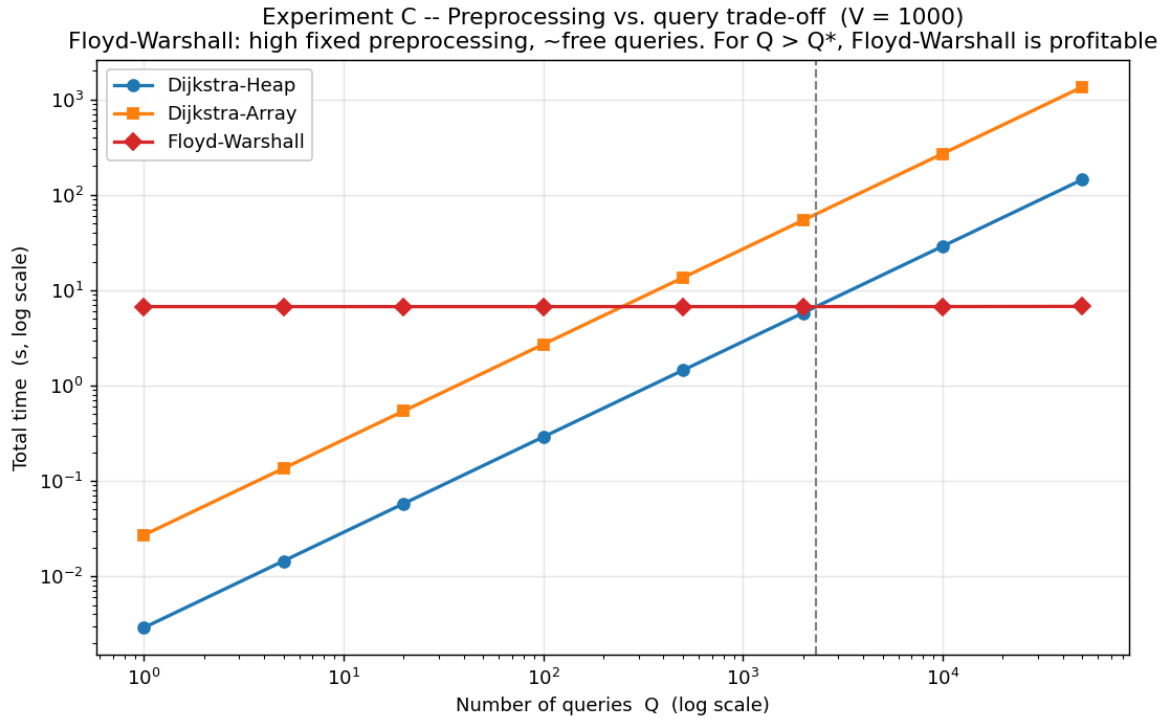


Figure 6: Experiment C: total time for Q queries (log-log). Floyd-Warshall's high fixed preprocessing is amortized when divided over many queries; after the break-even point it is the cheapest option.

Table 4: Measured unit costs and break-even point of Experiment C.

Quantity	Value
Dijkstra-Heap unit query	2.89 ms
Dijkstra-Array unit query	27.05 ms
Floyd-Warshall preprocessing	6.73 s
Floyd-Warshall unit query (matrix lookup)	0.7 μ s
Break-even point Q^* (Dijkstra-Heap vs. Floyd-Warshall)	≈ 2330 queries

Findings: Floyd-Warshall's preprocessing is hundreds of times the cost of a single Dijkstra query; but once the query count exceeds $Q \approx 2330$, Floyd-Warshall becomes cheaper overall. That is, the "heavy preprocessing" philosophy is rational in scenarios where the preprocessing cost can be divided over sufficiently many queries (e.g. continuous queries on an unchanging map); for a one-off query it is wasteful.

5.5. Experiment D — Real Road Network (DIMACS New York)

The real dataset is the New York City road network from the 9th DIMACS Implementation Challenge: 264,346 nodes, 733,846 directed arcs; the edge weights are real road distances. Connected subgraphs of controlled size were sampled from this network to compare the four approaches on real structure; in addition, scalability limits were tested on the full graph.

Experiment D -- Real road network (DIMACS New York)

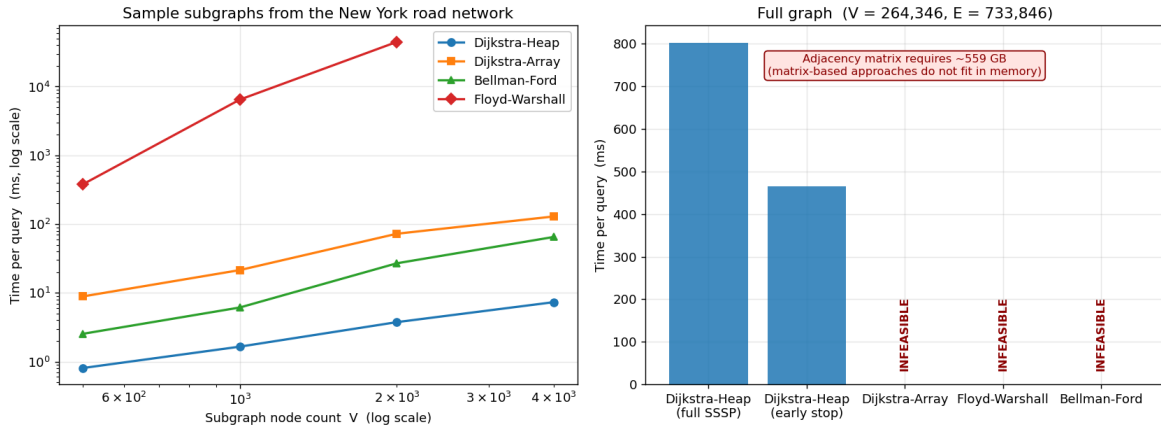


Figure 7: Experiment D: (left) scaling of the four approaches on sample subgraphs from the New York road network; (right) the full graph — only Dijkstra-Heap is feasible, the matrix-based approaches do not fit in memory.

Findings: On the subgraphs the ordering is consistent with the synthetic experiments; because road networks are very sparse and large-diameter, Dijkstra-Heap is clearly ahead. The truly striking result is on the full graph: Dijkstra-Heap answers an SSSP query over the entire network in 802.79 ms, with only 12.2 MB of peak memory. By contrast, Dijkstra-Array and Floyd-Warshall require an adjacency matrix; this matrix takes 559 GB and does not fit on any reasonable machine — that is, at this scale these two approaches are not merely slow but entirely infeasible. Bellman-Ford, in turn, is impractical because of its $O(V \cdot E) \approx 2e+11$ operations. This is the most concrete proof of the representation discussion in Section 2: the choice of representation can by itself determine whether an algorithm is feasible.

Early-stopping optimization: on the full graph, early-stopping Dijkstra took 466.04 ms for random (s, t) pairs. The gain is limited because random targets are on average "far"; early stopping provides a noticeable benefit mainly for nearby targets — the benefit of the on-demand optimization depends on the query distribution.

5.6. Memory Usage

Section 2 examined the memory of the graph representations. Here the algorithms' own working memory (excluding the representation) is compared.

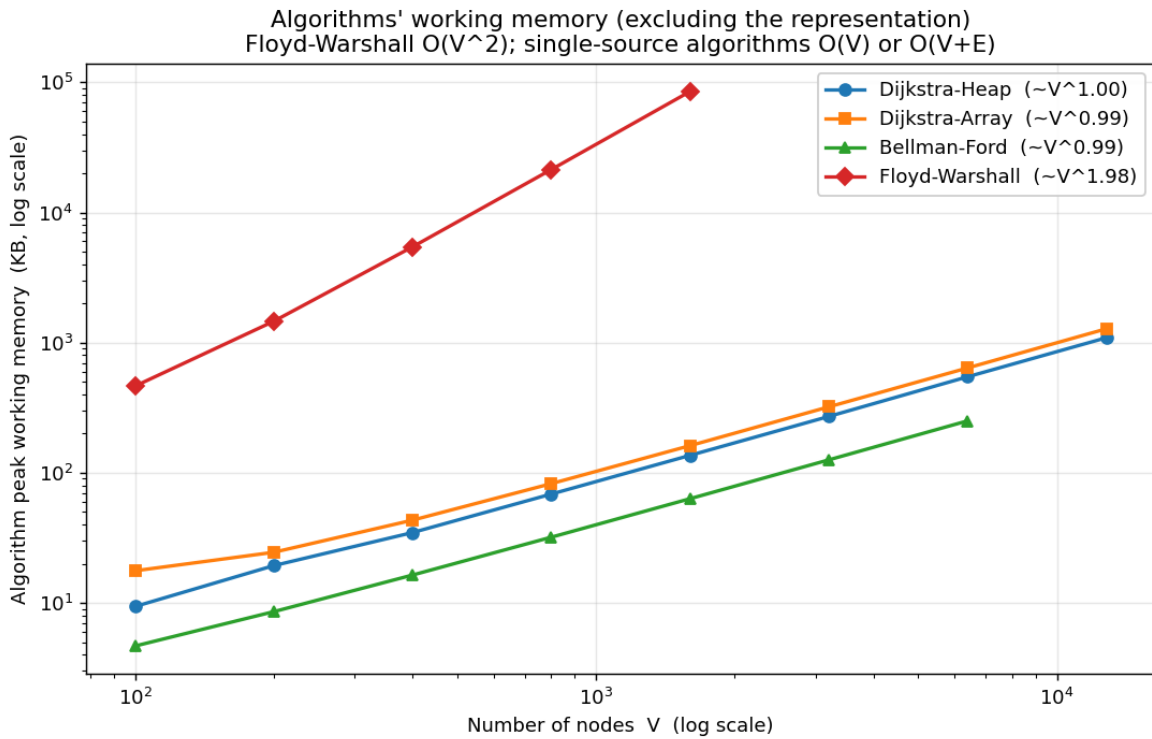


Figure 8: Peak working memory of the algorithms (excluding the representation). Floyd-Warshall keeps $O(V^2)$ matrices, while the single-source algorithms make do with $O(V)$ or $O(V+E)$.

Findings: Floyd-Warshall's working memory grows as $O(V^2)$ (the distance + path matrices); the single-source algorithms, on the other hand, use only $O(V)$ (the dist/pred arrays) or $O(V+E)$ (the heap). This is a memory ordering parallel to the time-complexity ordering, and it shows that the "memory vs. speed" trade-off is not actually a trade-off here — Floyd-Warshall is both slow and memory-heavy for a single query; its value lies only in the many-query scenario.

5.7. Comparing Theory with Experiment

Absolute times depend on hardware and on language/implementation constants. For a hardware-independent verification, the amount of work done (the number of relaxations / inspections) was also counted directly.

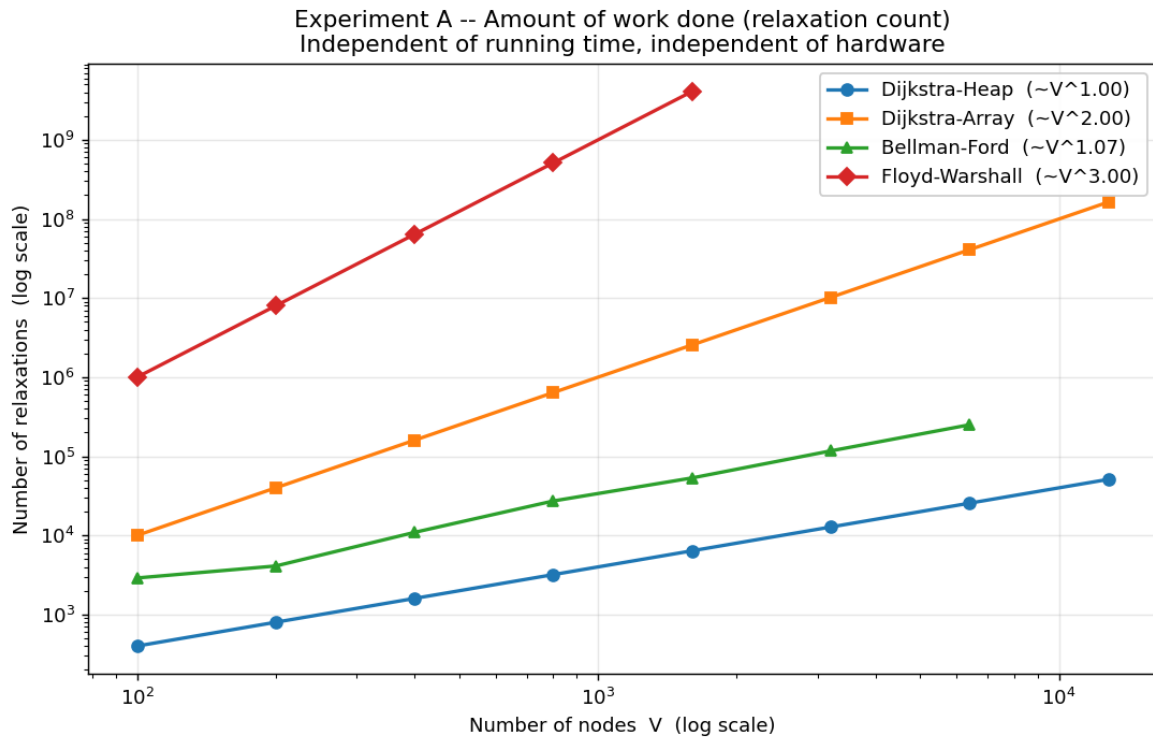


Figure 9: The amount of work done (number of relaxations) vs. V (log-log). This measurement is independent of hardware and implementation language; it directly reflects the theoretical complexity.

Table 5: Comparison of theory with experiment. The operation-count exponents match the theoretical exponents almost exactly (hardware-independent); the time exponents deviate because of constant factors and hardware effects.

Approach	Theoretical exponent (work)	Empirical exponent (op count)	Empirical exponent (time)
Dijkstra-Heap	≈ 1 (sparse: $E = O(V)$)	1.00	1.37
Dijkstra-Array	2 (V^2 cells inspected)	2.00	1.53
Bellman-Ford	$\approx 1-2$ (early stopping lowers it)	1.07	1.26
Floyd-Warshall	3 (V^3)	3.00	3.57

Findings: The operation-count (relaxation) exponents match the theoretical expectations almost exactly — exactly 2.00 for Dijkstra-Array; exactly 3.00 for Floyd-Warshall; ≈ 1 for Dijkstra-Heap and Bellman-Ford on sparse graphs. This proves, in a hardware-independent way, that the algorithms do exactly as much work as expected.

The time exponents, on the other hand, deviate from the theoretical exponents, and this deviation goes in two directions: (i) Dijkstra-Array's time exponent stays BELOW its work exponent (2.00) — because the fixed call overhead of the NumPy-vectorized operations is amortized better as the input grows; for small V this overhead dominates and pulls the log-log slope down. (ii) Floyd-Warshall's time exponent rises ABOVE its work exponent (3.00) — because the large V^2 matrices do not fit in the processor cache and memory bandwidth becomes a bottleneck. Conclusion: theoretical complexity gives the correct ORDERING; but the actual time is shaped by constant factors and hardware effects (cache, memory bandwidth). This

distinction between "theoretical complexity and empirical performance" is one of the project's core learning objectives.

6. Comparative Discussion

6.1. Which Approach Should Be Chosen When?

Table 6: An approach-selection guide by scenario. There is no single answer to "the best algorithm"; the answer depends on the input and the query pattern.

Scenario	Recommended approach	Rationale
Large, sparse graph; few queries	Dijkstra + Binary Heap	$O((V+E) \log V)$ and $O(V+E)$ memory; the only practical option
Small/medium, very dense graph	Dijkstra + Array Scan	$O(V^2) < O(E \log V)$; no data-structure overhead
Many (s,t) queries; the graph does not change	Floyd-Warshall	$O(V^3)$ preprocessing amortizes over many queries; query is $O(1)$
Negative edge weights are present	Bellman-Ford (or Floyd-Warshall)	Dijkstra's greedy assumption breaks; correctness is essential
A negative cycle is possible	Bellman-Ford / Floyd-Warshall	Both detect and report a negative cycle
The graph is unweighted (all edges equal)	BFS (not implemented in this project)	$O(V+E)$; weighted algorithms are needlessly expensive in this case

6.2. The Core Trade-offs

- **Preprocessing vs. query:** Floyd-Warshall pays a heavy $O(V^3)$ preprocessing and makes queries ~free; the others do no preprocessing but pay on every query. Experiment C measured the break-even point of these two philosophies.
- **Memory vs. access speed:** The adjacency matrix offers $O(1)$ edge access but requires $O(V^2)$ memory; the adjacency/edge list makes do with $O(V+E)$ memory but edge queries are more expensive. The real road network experiment showed that on large sparse graphs this trade-off sharpens to the point of "feasible / infeasible."
- **Speed vs. generality:** Bellman-Ford and Floyd-Warshall are asymptotically slower than Dijkstra; in return they solve negative weights correctly and detect negative cycles. A broader class of inputs comes at a higher cost.
- **Theoretical vs. empirical:** The empirical exponents came out close to the theoretical ones; but constant factors (NumPy vectorization, heap overhead, early stopping) can change the ordering on small inputs. Theory gives the direction; experiment gives the constants and the crossover points.

6.3. Which Assumptions Make an Algorithm Efficient/Inefficient?

The assumption that makes Dijkstra efficient is non-negative weights; if that assumption fails, Dijkstra does not merely slow down — it becomes incorrect (shown with a concrete example in the verification suite). The assumption that makes Dijkstra-Array and Floyd-Warshall efficient is that the graph is dense and relatively small; if the graph is large and sparse, the $O(V^2)$ memory makes these approaches infeasible. Bellman-Ford's early stopping kicks in under the assumption that the graph has a small diameter. And Floyd-Warshall's heavy preprocessing is meaningful only under the "many queries" assumption.

6.4. Other Approaches Not Implemented in This Project

For completeness, let us briefly mention other known solutions to the problem: (i) On unweighted graphs, BFS gives the shortest path in $O(V+E)$ — weighted algorithms are needlessly expensive in this case. (ii) On directed acyclic graphs (DAGs), topological sort + dynamic programming gives $O(V+E)$ and works even with negative weights. (iii) Johnson's algorithm solves the all-pairs problem on sparse graphs in $O(V \cdot E \log V)$, better than Floyd-Warshall. (iv) On road networks, methods such as A* or Contraction Hierarchies use preprocessing to greatly speed up queries. These methods are different combinations of the same four basic principles (relaxation, greedy selection, dynamic programming, preprocessing).

7. Conclusion

In this project, the problem of shortest-path queries on weighted graphs was implemented with four different algorithmic approaches — covering three graph representations and four design philosophies — analyzed theoretically, and compared experimentally on both synthetic and real data.

7.1. Summary of Findings

1. All four approaches are correct: verified with independent cross-checks, hand-computed examples, and path-reconstruction tests.
2. The general-purpose, scalable option is Dijkstra + Binary Heap; on large sparse graphs (including the 264-thousand-node real road network) it is the only practical option.
3. The "best algorithm" depends on the input: Dijkstra-Array stands out on dense graphs, Floyd-Warshall in the many-query scenario, and Bellman-Ford with negative weights.
4. The choice of graph representation is as decisive as the asymptotic complexity: $O(V^2)$ memory makes the matrix-based approaches entirely infeasible on large sparse graphs.
5. The measured empirical exponents are consistent with the theoretical complexities; theory and experiment confirm each other.

7.2. Lessons Learned

The project aimed at thinking like an algorithm designer. The core takeaway: the same problem has multiple valid solutions, and the right choice is made not with a memorized formula but by weighing conditions all together — the size and structure of the input, the query pattern, the memory budget, and the sign of the weights. Complexity analysis gives the direction of this decision; experimental measurement reveals the constants, the crossover points, and the feasibility limits. Together, the two turn a theoretical ordering into an engineering decision.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. — Chapters 22–25: graph algorithms and shortest paths.
- Dijkstra, E. W. (1959). "A note on two problems in connexion with graphs." *Numerische Mathematik*, 1, 269–271.
- Bellman, R. (1958). "On a routing problem." *Quarterly of Applied Mathematics*, 16, 87–90.
- Floyd, R. W. (1962). "Algorithm 97: Shortest Path." *Communications of the ACM*, 5(6), 345.
- Warshall, S. (1962). "A theorem on Boolean matrices." *Journal of the ACM*, 9(1), 11–12.
- 9th DIMACS Implementation Challenge — Shortest Paths (2006). USA-road-d.NY dataset.
<http://www.diag.uniroma1.it/challenge9/>
- Barabási, A.-L., Albert, R. (1999). "Emergence of scaling in random networks." *Science*, 286(5439), 509–512. — scale-free graph generator.

Appendix A: Running the Code

The project is written in pure Python (with only NumPy and Matplotlib as dependencies). The entire pipeline can be run with a single command:

```
bash run_all.sh          # the whole pipeline (verify→...→report→slides)

# or step by step:
python src/verify.py      # correctness tests
python src/benchmark.py --full # experiments -> results/raw/*.json
python src/plots.py       # plots + tables -> results/
python src/animate.py     # visual simulations -> results/animations/
python src/make_report_en.py # this report -> report/Report.docx
python src/make_slides_en.py # presentation -> presentation/Presentation.pptx

# live demo (can be run during the presentation):
python src/demo.py        # compare the four algorithms side by side
python src/demo.py --kind grid --n 25 --trace # small graph + step-by-step trace
```

Directory structure:

```
src/algorithms/  separate implementations of the four approaches
src/graph.py    the three graph representations
src/generators.py synthetic graph generators
src/data_loader.py DIMACS real-data reader + subgraph sampler
src/verify.py   correctness test suite
src/benchmark.py experiment runner
src/plots.py    plot + table generator
src/animate.py  visual simulation (GIF + snapshot) generator
src/demo.py     live comparison demo (command line)
data/ results/ report/ presentation/ (inputs and outputs)
```

All numbers in this report were produced from the following run: profile "full", seed 20252, date 2026-05-14T14:58:27, total benchmark time 463 s. The results are reproducible with the same seed.